Ex – 01	Exploring Matplotlib, Seaborn, and Plotly

AIM

To explore and perform data visualization using Matplotlib, Seaborn, and Plotly by loading a suitable dataset, preprocessing the data, creating different types of plots, and interpreting the insights obtained from each visualization.

PROBLEM STATEMENT

To analyze a suitable dataset using Python visualization libraries such as Matplotlib, Seaborn, and Plotly. The dataset is checked for missing values, column names, and dimensions. Unimportant features and missing values are removed, and Bar Plot, Scatter Plot, Pie Chart, Histogram, and Box Plot are created to identify patterns, distributions, relationships, and outliers. Finally, the visualization libraries are compared based on their suitability for different analytical scenarios.

PROCEDURE

1. Import the required Python libraries: Pandas, Matplotlib, Seaborn, and Plotly.
2. Load the built-in Iris dataset into a Pandas DataFrame.
3. Check for missing values using the isnull() function.
4. Display all column names using the columns attribute.
5. Find the number of rows and columns using the shape attribute.
6. Remove rows containing missing values, if any.
7. Remove unimportant features, such as the flower identifier, if present.
8. Create a Bar Plot to compare the average petal length of different Iris species.
9. • Create a Scatter Plot to examine the relationship between sepal length and petal length.
10. • Create a Pie Chart to display the proportion of each Iris species.

```
# 1. Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
from sklearn.datasets import load_iris
```

```
# 2. Load the Iris dataset
iris = load_iris(as_frame=True)
df = iris.frame
```

```
# Add flower species names
df["species"] = df["target"].map(
    dict(enumerate(iris.target_names))
)
```

```
# 3. Check for missing values
print("Missing Values:")
print(df.isnull().sum())
```

```
Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target               0
species              0
dtype: int64
```

```
# 4. Display column names
print("\nColumn Names:")
print(df.columns.tolist())
```

```

Column Names:
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)', 'target', 'species']
```

```
# 5. Display dataset shape
print("\nDataset Shape:", df.shape)
```

```

Dataset Shape: (150, 6)
```

```
# 6. Remove rows containing missing values
df.dropna(inplace=True)
```

```
# 7. Remove unimportant feature (target is redundant
# because species contains the class names)
df.drop(columns=["target"], inplace=True)
```

```
print("\nCleaned Dataset Shape:", df.shape)
print("\nDataset Information:")
print(df.head())
```

```

Cleaned Dataset Shape: (150, 5)

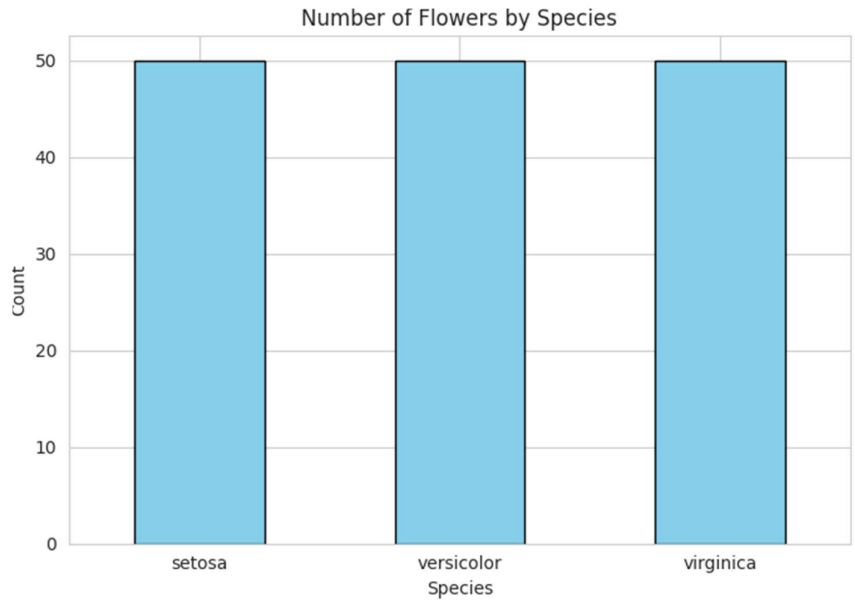
Dataset Information:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

  species
0  setosa
1  setosa
2  setosa
3  setosa
4  setosa
```

```
# Set Seaborn style
sns.set_style("whitegrid")
```

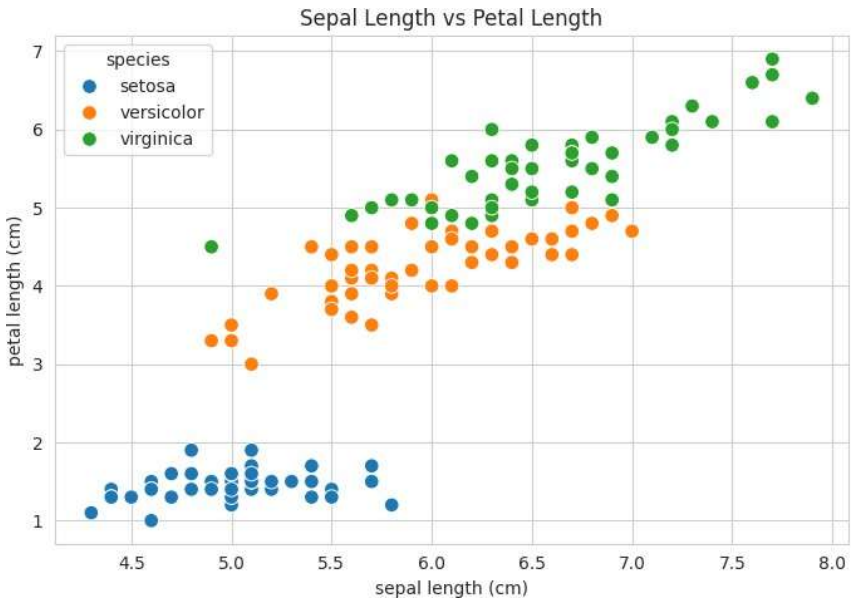
BAR PLOT

```
# 8(a). BAR PLOT using Matplotlib
plt.figure(figsize=(7, 5))
df["species"].value_counts().plot(
    kind="bar", color="skyblue", edgecolor="black"
)
plt.title("Number of Flowers by Species")
plt.xlabel("Species")
plt.ylabel("Count")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



SCATTER PLOT

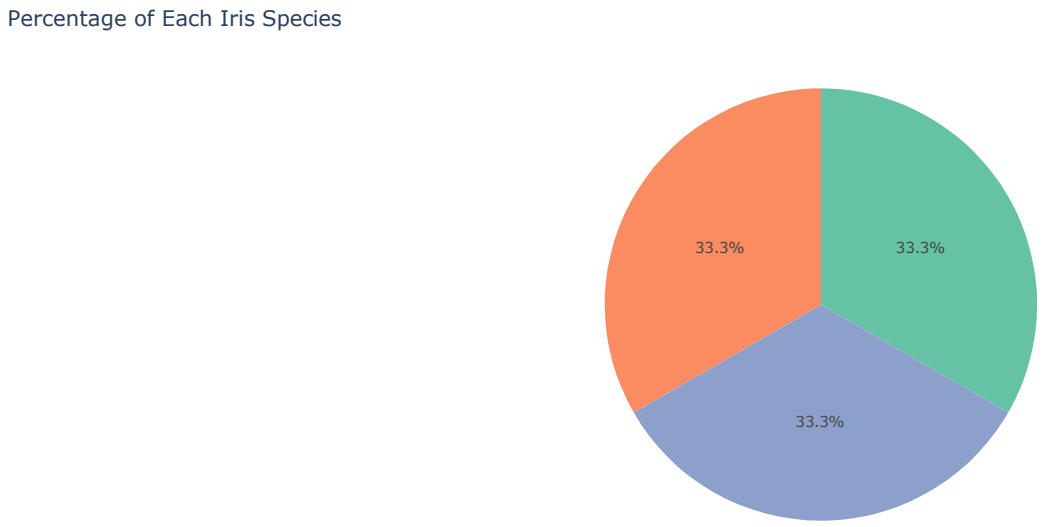
```
# 8(b). SCATTER PLOT using Seaborn
plt.figure(figsize=(7, 5))
sns.scatterplot(
    data=df,
    x="sepal length (cm)",
    y="petal length (cm)",
    hue="species",
    s=70
)
plt.title("Sepal Length vs Petal Length")
plt.tight_layout()
plt.show()
```



- ## PIE CHART

```
# 8(c). PIE CHART using Plotly
species_counts = df["species"].value_counts().reset_index()
species_counts.columns = ["species", "count"]

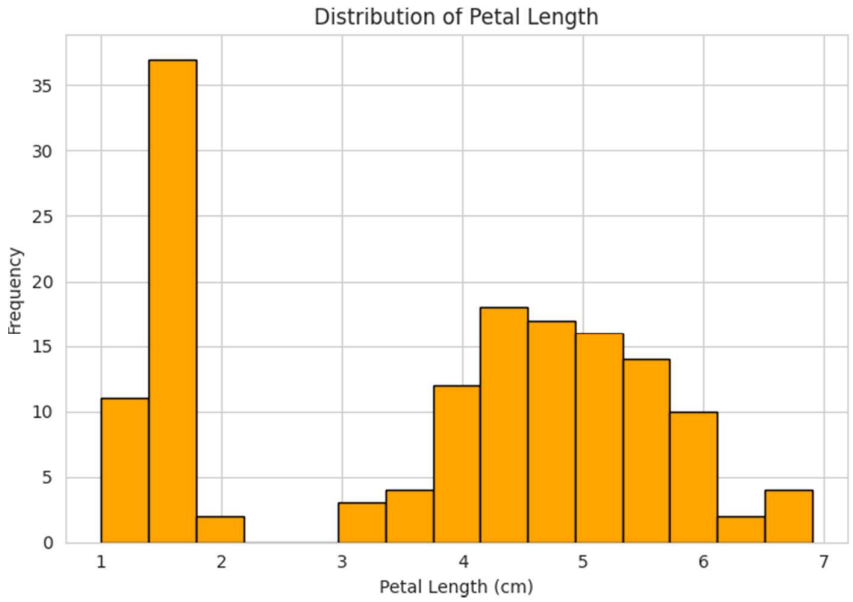
fig = px.pie(
    species_counts,
    names="species",
    values="count",
    title="Percentage of Each Iris Species",
    color_discrete_sequence=px.colors.qualitative.Set2
)
fig.show()
```



- ## HISTOGRAM bold text

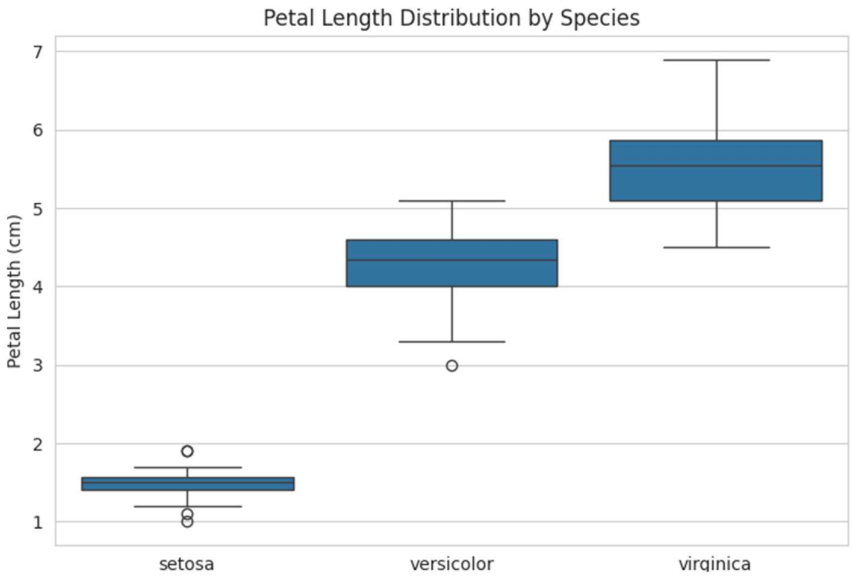
```
# 8(d). HISTOGRAM using Matplotlib
plt.figure(figsize=(7, 5))
plt.hist(
    df["petal length (cm)"],
```

```
    bins=15,
    color="orange",
    edgecolor="black"
)
plt.title("Distribution of Petal Length")
plt.xlabel("Petal Length (cm)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```



° BOX PLOT

```
# 8(e). BOX PLOT using Seaborn
plt.figure(figsize=(7, 5))
sns.boxplot(
    data=df,
    x="species",
    y="petal length (cm)"
)
plt.title("Petal Length Distribution by Species")
plt.xlabel("Species")
plt.ylabel("Petal Length (cm)")
plt.tight_layout()
plt.show()
```



Insights obtained

1. Bar Plot: Compares the average petal length of the three Iris species. Setosa has the smallest average petal length, while Virginica has the largest.
2. Scatter Plot: Shows the relationship between sepal length and petal length. Virginica generally has larger petals, while Setosa forms a distinct cluster.
3. Pie Chart: Shows that each species represents approximately 33.3% of the dataset because all three species contain 50 observations.
4. Histogram: Displays the distribution of sepal length and helps identify common value ranges and differences between species.
5. Box Plot: Compares petal-length distributions and highlights variation, median values, and potential outliers within each species.

RESULT

The Iris dataset was successfully loaded, checked for missing values, and preprocessed using Pandas. The dataset contains 150 rows and 5 columns before preprocessing, with no missing values in the standard dataset.

The five visualizations were created successfully using Matplotlib, Seaborn, and Plotly.